

Space Invaders Design Document

Iterator Pattern

A. Challenge in Space Invaders Game

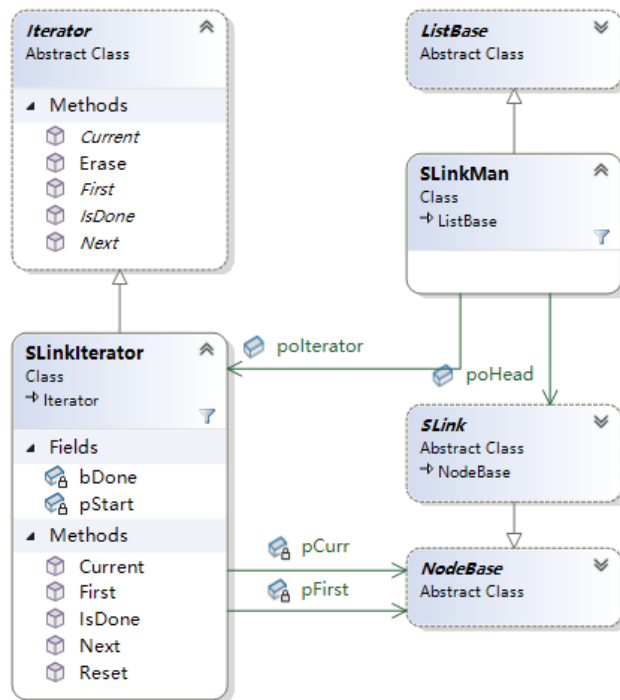
Problem: In our system, we needed a way to iterate over different types of collections in a uniform manner. Some collections store elements in linked lists, while others use trees or arrays. The challenge was that iterating over each collection type required different logic, making the code complex and inconsistent. Additionally, most code using these collections does not need to know the underlying storage structure—only how to traverse and access elements.

Solution: We implemented the **Iterator Pattern**, which encapsulates the iteration logic within dedicated iterator objects. Instead of manually handling pointers or indexing, we introduced a standard interface for iterating over collections. This interface provides methods to access the first element, the current element, the next element, and check if the iteration is complete. By doing so, any collection can be traversed in a simple and consistent manner, regardless of how it is stored internally.

B. Pattern Description

Intent: The **Iterator Pattern** provides a standardized way to traverse collections without exposing their internal structure. This allows loops to be written in a clean and generic way, improving code readability and maintainability. By using an iterator object, the logic for iterating over a collection is encapsulated and can be swapped out as needed without modifying the core iteration logic.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: The iteration logic is contained within the iterator objects, preventing external code from handling low-level traversal details.

Delegation: The iterator manages the iteration, while the collection provides access to its elements without exposing its internal structure.

Abstraction: The base **Iterator** class enforces a standardized interface with methods like **Current()**, **Erase()**, **First()**, **IsDone()**, and **Next()**, enabling uniform traversal across various collection types.

D. Usage in Space Invaders

The **Iterator Pattern** plays a key role in managing collections in the Space Invaders game. Instead of manually iterating over different structures, we utilize iterators to abstract the traversal logic.

Sprite Batch Rendering: The **Sprite Batch Manager** iterates over collections of **Sprite Batch** using iterators, ensuring that each sprite node is drawn efficiently.

Timer Event Processing: The **Timer Event Manager** iterates over scheduled events, triggering them at the appropriate time.

Game Object Hierarchy Processing: The **Composite Iterator** and **Reverse Composite Iterator** navigate the game object hierarchy, ensuring that each **Game Object** undergoes consistent updates and collision checks.

By using iterators, the game can process various collections efficiently without requiring knowledge of their internal structure. This makes the codebase more flexible and maintainable while allowing new iterator types to be added when needed.

Strategy Pattern

A. Challenge in Space Invaders Game

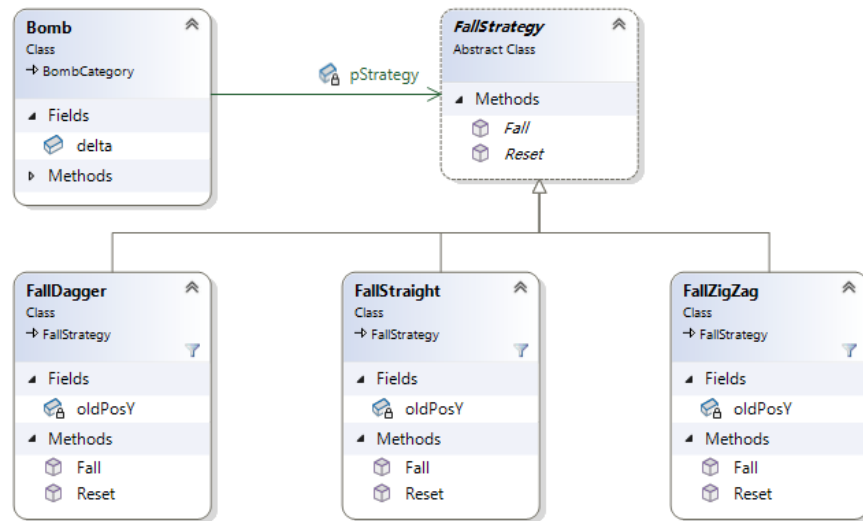
Problem: In Space Invaders, bombs need to exhibit three distinct falling behaviors, each with different visual effects. These behaviors should be independent of one another and interchangeable, allowing random selection when a bomb enters the scene. Implementing this as an animation is impractical, so we need a procedural approach that is easy to modify and extend.

Solution: To introduce specialized behaviors for bomb movement in *Space Invaders*, we implemented the **Strategy Pattern**. We created a **FallStrategy** base class that defines a generic contract for fall behaviors, primarily through a **Fall()** method. Each unique falling effect is implemented as a concrete subclass of **FallStrategy**, manipulating the bomb sprite in distinct ways to create varied visual effects.

B. Pattern Description

Intent: The **Strategy Pattern** encapsulates behaviors into separate objects and references them through a common interface, enabling dynamic interchangeability without modifying the core logic of the object using them. This approach allows the system to **automatically specialize behaviors**, making it adaptable and modular. By implementing the **Strategy Pattern**, we ensure that: Behaviors are **isolated**, making them easy to modify and extend. The system remains **organized**, eliminating special-case logic. **New fall behaviors** can be added seamlessly without altering existing code.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: Each fall behavior is encapsulated in its own strategy class, ensuring that the logic is isolated and independent.

Composition Over Inheritance: Instead of using multiple subclasses for different bomb types, we use composition, allowing the behavior to be changed dynamically.

Abstraction: The **FallStrategy** base class defines a common interface through the **Fall()** method, ensuring that all derived strategies adhere to a consistent structure. This design simplifies the addition of new behaviors while maintaining a cohesive and extensible system.

D. Usage in Space Invaders

Automatic Behavior Variation: The system automatically selects a falling behavior for each bomb, ensuring diverse gameplay experiences.

Different Drawing Behaviors: Each strategy alters the bomb's appearance as it falls, adding variety.

Elimination of Special Cases: The Strategy Pattern removes the need for hardcoded conditional logic, simplifying the system.

Easy Extension: New fall behaviors can be introduced by simply adding a new strategy class without modifying existing code.

Proxy Pattern

A. Challenge in Space Invaders Game

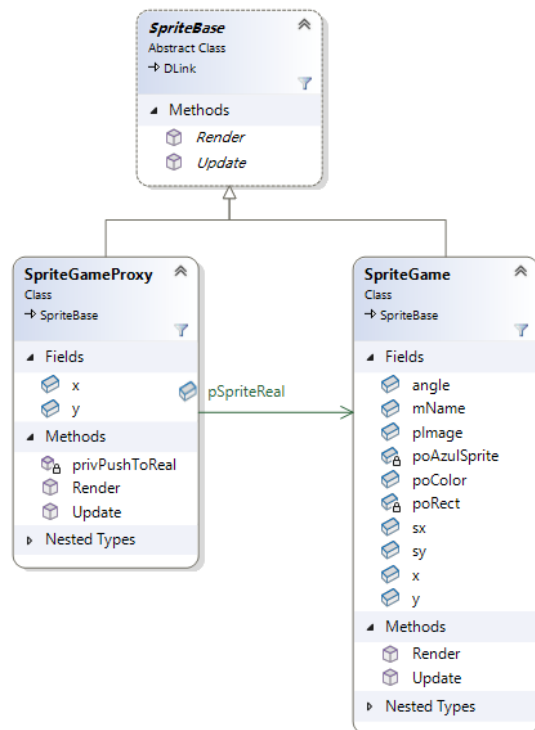
Problem: In our Space Invaders game, we faced a challenge in efficiently managing multiple sprite objects. Many game objects need to use the same Sprite but with different positional data. However, the base **SpriteGame** class forces each object to maintain a separate copy of the entire sprite, even when only a few properties—such as position—need to change.

Solution: We implemented the Proxy Pattern by creating **SpriteGameProxy**. This proxy class holds a reference to a single instance of **SpriteGame** and only stores the position-related data (e.g., x, y). This approach enables the creation of multiple "instances" of a sprite while avoiding redundant duplication. Additionally, it ensures efficient rendering by delegating update operations to the actual **SpriteGame** object.

B. Pattern Description

Intent: The Proxy Pattern introduces an intermediary layer to **SpriteGame**, allowing optimized memory and processing use by deferring heavy operations to the real object when necessary. The proxy object, in this case **SpriteGameProxy**, holds a reference to the actual **SpriteGame**, modifying or delegating requests as needed.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: **SpriteGameProxy** hides the real **SpriteGame** object while exposing only essential methods like **Render()** and **Update()**.

Delegation: **SpriteGameProxy** does not perform rendering itself—it delegates this responsibility to **SpriteGame**, ensuring updates happen in one place only.

Abstraction: **SpriteBase** ensures that both **SpriteGame** and **SpriteGameProxy** follow a common interface, keeping the system flexible and extendable.

D. Usage in Space Invaders

While many of its methods delegate calls directly to **SpriteGame**, **SpriteGameProxy** exposes only the properties and functionalities relevant to its upper-level callers. This ensures a controlled and streamlined interface, reducing complexity and preventing unintended access to internal details of the **SpriteGame** object.

Memory Optimization: Instead of storing multiple full-fledged **SpriteGame** instances, **SpriteGameProxy** maintains lightweight references and only position-related properties, reducing redundant instantiations and saving memory.

Lazy Initialization: The real **SpriteGame** object is only accessed when necessary (e.g., in the render function during the draw process in **SpriteBatchMan**), avoiding unnecessary computations during idle frames.

Efficient Rendering and Updates: The proxy modifies only position-related properties (e.g., x, y), pushing them to the real **SpriteGame** through **privPushToReal()**. This minimizes redundant updates, ensuring smooth performance.

Factory Pattern

A. Challenge in Space Invaders Game

Problem: In Space Invaders, the initialization of Aliens involves a complex series of steps, including:

- Resurrecting objects from a Ghost Manager.
- Constructing new objects with specific parameters.
- Attaching them to the game's rendering and updating systems.

- Creating different hierarchical layers of game objects (individual sprite, column, grid, or root) and organize them into a structured tree layout.

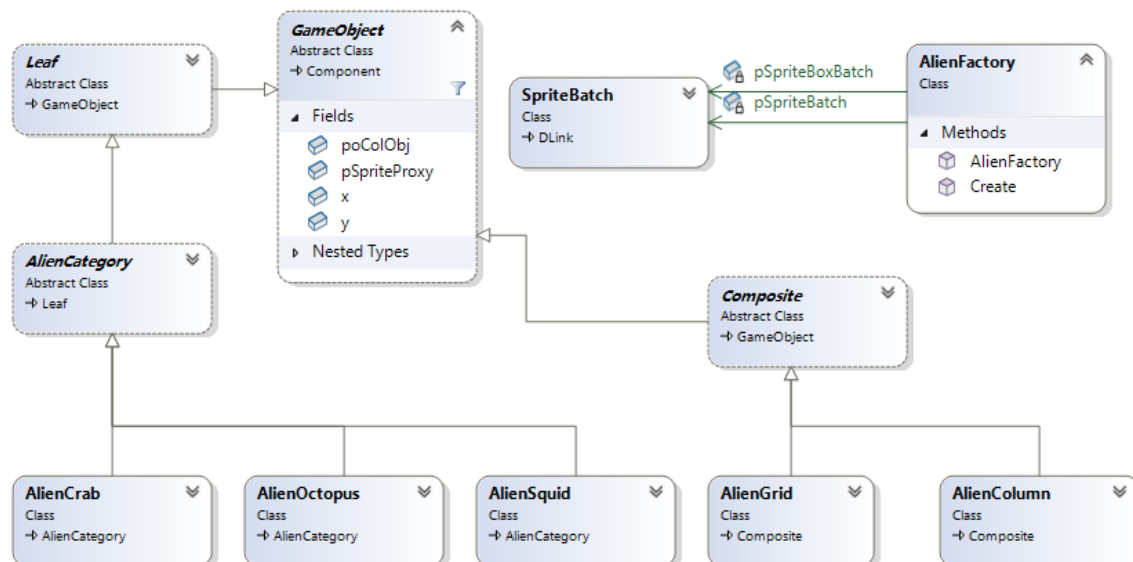
This process must be repeated every time Aliens are created, increasing the risk of bugs and inconsistencies if not managed carefully.

Solution: To address this, we implemented the **Factory Pattern** by introducing an **Alien Factory**. This factory encapsulates the entire creation and setup process for Aliens, abstracting away the intricate details. By providing reusable methods, the factory ensures consistent and error-free initialization of Aliens. Additionally, it allows the grid of Aliens to be generated or regenerated with a single method call, ensuring uniformity across the game. This approach eliminates the need for other parts of the code to understand how Aliens are created, improving maintainability and adaptability.

B. Pattern Description

Intent: The **Factory Pattern** simplifies object creation by encapsulating the construction logic. It acts as an abstraction layer between the code using the object and the object creation process, ensuring that objects are always in a valid state before use. In *Space Invaders*, the Factory Pattern ensures that Aliens are properly resurrected or created, attached to game systems, and placed in the grid, without exposing the underlying complexity to the rest of the game.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: The **Alien Factory** encapsulates the complex creation and setup logic for Aliens, isolating it from the rest of the game code. This reduces the risk of errors and makes the system easier to manage.

Composition Over Inheritance: Instead of relying on inheritance, the Factory uses composition to handle the setup process. This approach makes it easier to modify or extend the initialization logic without impacting other parts of the system.

Abstraction: The Factory serves as an abstraction layer, providing a simple interface for creating Aliens while hiding the underlying complexity. This ensures that no other code needs to know how Alien Game Objects are created or initialized.

D. Usage in Space Invaders

The **Alien Factory** in *Space Invaders* simplifies and organizes the complex creation process of Aliens. The Factory provides several methods that ensure consistent and reliable initialization.

Aliens are always created or resurrected in the same way, either by constructing new objects or retrieving them from the **Ghost Manager**. Each Alien is equipped with the necessary Sprites (**Game Sprite** and **Collision Box Sprite**), which are attached to the Sprite Batches passed to the Factory.

The initialization logic is isolated within the Factory, meaning any changes to how Aliens are created or set up can be made in one place. This makes the system highly maintainable and adaptable. This ensures that other parts of the system don't need to know how Aliens are initialized, making the overall architecture cleaner and more focused on the game mechanics rather than on managing object creation.

State Pattern

A. Challenge in Space Invaders Game

Problem: In *Space Invaders*, the player ship exhibits distinct behaviors based on whether it has fired a missile or if it hits a side bumper:

- **Missile Behavior:** If a missile is in mid-air, the ship cannot fire another missile, as only one missile is allowed on the screen at a time. Once the missile hits an Alien or reaches the top of the screen, the ship is free to fire again.

- **Bumper Behavior:** If the ship collides with either the left or right bumper, its movement is restricted, and it can only move in the opposite direction. However, if the ship is positioned between the two bumpers, it can move freely in both left and right directions.

This dynamic behavior requires the ship to switch between states (ready to fire or not ready, able to move in both directions or only one direction) based on the missile's status. Managing these transitions with conditional logic would lead to complex and error-prone code, making it challenging to maintain and extend the game's functionality.

Solution: To address this, we implemented the **State Pattern**. The player ship is represented by two primary states for firing behavior:

1. **Ready to Fire:** The ship can fire a missile.
2. **Not Ready (Missile in Flight):** The ship cannot fire until the missile is no longer active.

Additionally, we incorporate a **Move State** for handling directional movement:

1. **Move Both Directions:** The ship can move left and right when positioned between the two side bumpers.
2. **Move Right/Left Direction:** If the ship hits a bumper (left or right), its movement is restricted to only the opposite direction.

The ship transitions between these states dynamically. These transitions are managed by state classes (**ShipMissileReady**, **ShipMissileFlying**, **ShipMoveLeft**, **ShipMoveRight**, and **ShipMoveBoth**), each of which encapsulates the behavior and logic for its respective state, ensuring clean and maintainable code for both firing and movement actions.

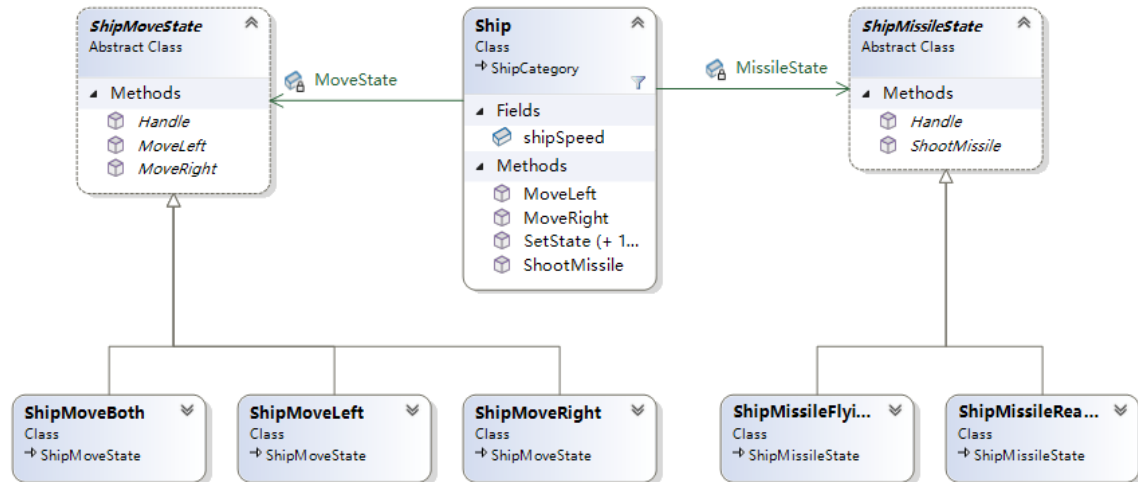
Each state type inherits from a common abstract base class, **ShipMissileState**, which defines the shared methods and behaviors for firing, and **ShipMoveState**, which defines the shared methods and behaviors for movement across different states. These base classes provide a common structure for firing and movement actions, allowing for flexibility and extensibility. Each specific state can override or implement the appropriate behavior while maintaining consistency across the system.

B. Pattern Description

Intent: The **State Pattern** models a finite state machine, enabling an object to alter its behavior dynamically based on its internal state. In *Space Invaders*, the State Pattern ensures that the player ship's missile-firing and movement behaviors are controlled by its current state. Each state encapsulates specific behaviors, and transitions between

states define how the ship responds to events such as firing a missile or the missile exiting the screen. This pattern is especially beneficial for avoiding complex conditional logic, simplifying transitions, and maintaining modular, extensible code that is easy to modify or extend in the future.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: The behavior of the player ship is encapsulated within distinct state classes. Each state manages its own logic for firing missiles or movement, as well as transitioning to other states, ensuring modular and maintainable code.

Delegation: The player ship delegates its firing or movement behavior to the current state object. This allows the state to control whether the ship can fire or which direction it is allowed to move, simplifying the ship's logic and making the code more manageable.

Abstraction: **ShipMoveState** and **ShipMissileState**, provide the abstraction for all state behaviors, defining a common interface for firing, movement, and state transitions.

ShipMoveState manages the movement behavior, while **ShipMissileState** handles the missile firing behavior. This separation of concerns ensures that each state can focus on its specific responsibilities, making the system more modular and easier to extend.

D. Usage in Space Invaders

Dynamic Behavior: The ship's ability to fire a missile is dynamically controlled by its current state. In the **ShipMissileReady**, the ship can fire a missile. Once fired, the ship transitions to the **ShipMissileFlying**, where it cannot fire again until the missile hits the ceiling or an Alien. Upon either of these events, the ship transitions back to the

ShipMissileReady. When the ship collides with a bumper, its movement state switches between the three movement states—**ShipMoveLeft**, **ShipMoveRight**, and **ShipMoveBoth**—depending on which bumper is hit. If the ship is positioned between the bumpers, it can move freely in both directions.

State Transitions: Transitions between states, such as **ShipMissileReady** and **ShipMissileFlying**, are managed by the state classes themselves. This approach eliminates the need for complex conditional logic in the main game loop, simplifying the code structure and improving.

Visitor Pattern

A. Challenge in Space Invaders Game

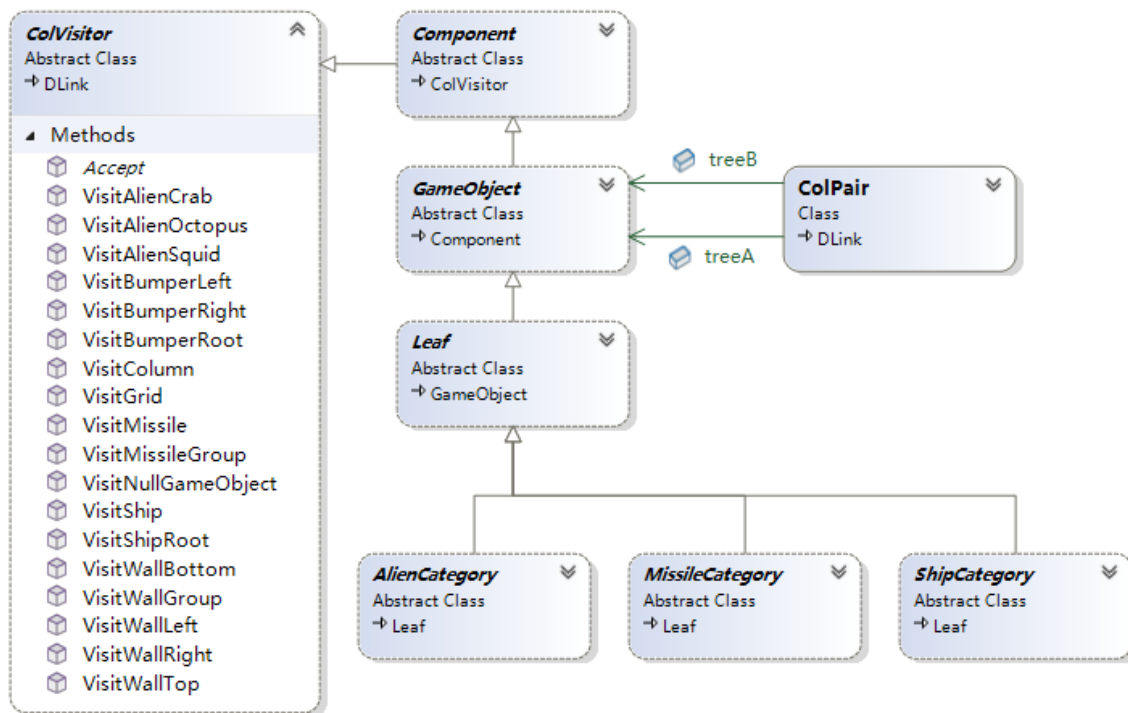
Problem: In our Space Invaders game, efficiently detecting collisions between game objects presented a significant challenge. Checking all possible object pairs for collisions is computationally expensive and unnecessary, as many objects will never interact. Instead of performing costly, exhaustive collision checks, we needed a system that only processes relevant interactions while abstracting different object interactions.

Solution: We implemented the **Visitor Pattern** to streamline collision detection. This approach abstracts all possible interactions between objects, including both relevant and irrelevant collision pairs. However, the system only implements the visitor for object pairs that are known to interact, avoiding unnecessary collision checks. By filtering out irrelevant collisions at the design level, we significantly reduce the computational overhead.

B. Pattern Description

Intent: The Visitor Pattern enables dynamic dispatch for collision handling, allowing game objects to define their interactions without relying on excessive conditionals or type-checking logic. It ensures that each game object processes only the collisions that matter.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: The Visitor Pattern encapsulates collision logic within dedicated **Visit()** methods, keeping collision handling separate from the game objects themselves. This modular approach improves maintainability.

Delegation: Each game object delegates its collision handling to the visitor system, allowing it to process only relevant interactions. This prevents unnecessary checks and ensures efficiency.

Abstraction: The **ColVisitor** base class provides an abstraction layer, ensuring a consistent interface for collision interactions. Objects only need to implement interactions that are meaningful, skipping irrelevant ones.

D. Usage in Space Invaders

Optimized Collision Handling: Rather than checking all possible object pairs, the Visitor Pattern ensures that only predefined collision pairs are considered. To further optimize collision handling, we introduced a **Collision Pair Detection Manager** class (**ColPairMan**). This class uniformly manages collision detection across the game, decoupling it from other game modules. By using the **Visitor Pattern**, **ColPairMan** ensures that only

predefined collision pairs are evaluated, rather than checking all possible object combinations. For example:

- A **Missile** colliding with an **Alien** triggers destruction.
- The **Alien Grid** hitting a Side Wall causes it to reverse direction and move downward.
- A **Missile** and a **Bumper** do not interact, so no collision check occurs.

Performance Gains: Instead of checking all object pairs, we only process relevant collisions.

Modular & Scalable: New collision interactions can be added by extending the visitor, without modifying the core game loop.

No Dynamic Type Casting: The pattern leverages polymorphism, avoiding expensive type-checking operations.

By applying the **Visitor Pattern**, Space Invaders efficiently detects collisions while maintaining a clean and extensible architecture.

Observer Pattern

A. Challenge in Managing Game Reactions

Problem: In Space Invaders, when specific game events occur—such as a missile hitting an alien—multiple reactions need to be triggered. For example:

- When an **Alien** is destroyed, the game should play a sound, increase the player's score, and generate a death explosion effect.
- When a **Missile** collides with an object, it should be removed from the scene, and the player should regain the ability to fire.

Manually managing these reactions through direct function calls would make the code complex and difficult to maintain. We needed a solution that would decouple event detection from event reactions, ensuring flexibility and extensibility.

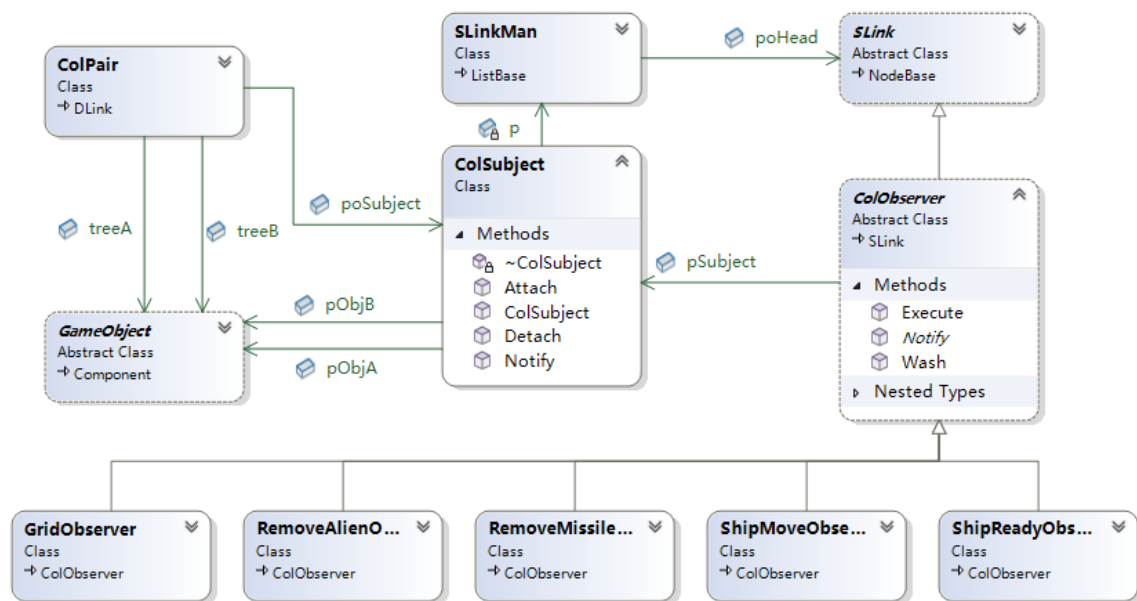
Solution: To address this, we implemented the **Observer Pattern**, which provides a modular and scalable way to handle game reactions. In our system, each collision event is associated with a **Subject** object. This Subject can register multiple **Observers**, each responsible for handling specific game mechanics. When a collision occurs, the Subject

notifies all registered Observers, enabling them to execute their respective actions independently. This design decouples the collision detection logic from the reactions it triggers, ensuring that new behaviors can be added or modified without altering the core collision system. The result is a flexible and maintainable architecture that supports dynamic and complex interactions.

B. Pattern Description

Intent: The Observer Pattern introduces a **decoupled event-handling mechanism**, ensuring that game objects do not need to directly manage event responses. Instead, event handlers (**Observers**) subscribe to relevant game events (**Subjects**), reacting only when notified. This approach allows for easy modifications and extensions to game mechanics without altering core logic. This structure allows for clean separation between **collision detection** and **game responses**, making the system more maintainable and adaptable.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: The Observer Pattern isolates event handling logic from core game mechanics. Observers manage their own reactions, while Subjects only handle event broadcasting.

Delegation: Subjects do not directly execute event responses. Instead, they **delegate reactions** to Observers, ensuring modular and reusable event handling.

Abstraction: A **Collision Subject** provides a unified interface for managing event notifications, while **Observers** implement specific reactions. This abstraction simplifies game logic and allows easy expansion.

D. Usage in Space Invaders

Optimized Collision Handling:

The **Observer Pattern** ensures that game objects only react to relevant events, improving efficiency and maintainability.

How It Works:

1. The collision system identifies a collision pair (e.g., **Alien** vs. **Missile**).
2. A **Collision Subject** associated with the event triggers a **Notify()** method.
3. Any **Observer** subscribed to the event receives a notification and executes the appropriate game behavior (e.g., removing objects, updating scores, playing sounds).
4. Multiple Observers can be attached to a single event, ensuring all necessary reactions occur.

By leveraging the **Observer Pattern**, Space Invaders efficiently manages game events, enabling a highly extensible and maintainable architecture.

Command Pattern

A. Challenge in Managing Timed Events

Problem: In Space Invaders, certain game actions need to be triggered at scheduled times. These time-based events include:

- **Alien movement** at fixed intervals.
- **Ship respawn delays** after destruction.
- **Animation timing** for visual effects.

Since different events require timing, we needed a unified system to manage them efficiently. The solution had to support **one-off events** that execute once and **recurring**

events that re-register themselves. Additionally, checking all possible events every frame would be inefficient, so an optimized scheduling system was necessary.

Solution: We implemented the **Command Pattern** combined with a **Timer Event System** to manage scheduled events efficiently. Each **Command object** is encapsulated within a **Timer Event**, defining its behavior through an **Execute()** method. These Timer Events are then registered with a **Timer Event Manager**, which maintains a **sorted list** of events based on their scheduled execution time.

During the game loop, the Timer Event Manager:

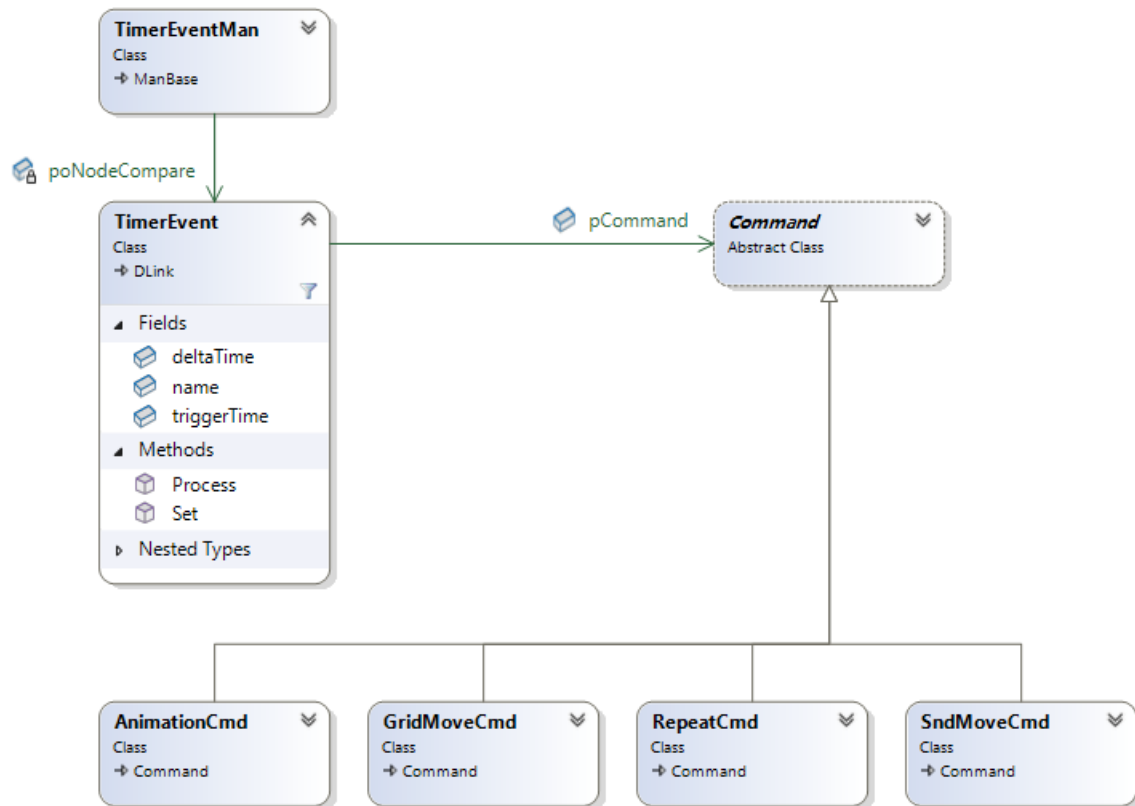
1. Iterates through the scheduled events.
2. Checks if the current time has reached the event's execution time.
3. If the time matches, it triggers the associated **Command's Execute()** method.
4. If the event is recurring, it re-registers itself with a new execution time.

By maintaining a **sorted list**, the system efficiently processes only the events that are due, skipping unnecessary checks (**early out optimization**). This approach allows different game mechanics to uniformly utilize timed events while keeping the system modular and scalable.

B. Pattern Description

Intent: The **Command Pattern** encapsulates game actions as objects, allowing them to be scheduled, stored, and executed at the right time. By integrating it with a **Timer Event System**, we create a **flexible, reusable, and efficient** way to handle all time-based events in the game.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: Each scheduled action is wrapped in a Timer Event, which contains a Command, keeping execution logic separate from the Timer system.

Delegation: The **Timer Event Manager** maintains and triggers **Timer Events** without needing to know the specifics of each Command's behavior.

Abstraction: The **Command** base class provides a unified interface, allowing any timed action to be executed through its **Execute()** method.

Optimization: **Timer Events** are sorted by execution time, enabling early exit checks and improving performance by processing only necessary events each frame.

D. Usage in Space Invaders

Efficient Timed Event Execution: Rather than evaluating all scheduled events every frame, the system processes only relevant events based on execution time.

An **Alien Movement** event triggers at its scheduled time and re-registers itself for continuous motion.

A **Ship Respawn** event executes after a delay and does not re-register, ensuring it runs only once.

An **Animation** event plays a single time when triggered.

By maintaining a sorted event queue, the system efficiently exits early in the update loop, significantly improving performance. Since **Commands** are encapsulated within **Timer Events**, any new time-based action can be integrated without modifying the core **Timer** logic, ensuring scalability and adaptability.

Composite Pattern

A. Challenge in Managing Game Object Hierarchy

Problem: In Space Invaders, the game scene consists of numerous **Game Objects** with a hierarchical structure. Some Game Objects naturally form groups, such as:

- **Alien Sprites**, which move as a single unit in a 2D grid.
- **Shield Bricks**, which are grouped together and need to be replicated efficiently.

Manually managing these objects individually would lead to complex, hard-to-maintain code. Additionally, a **hierarchical organization** could optimize the **collision detection system**, reducing unnecessary checks by grouping objects into manageable subsets. We needed a solution that allowed objects to be grouped, managed, and operated on collectively while maintaining flexibility for hierarchical relationships.

Solution: We implemented the **Composite Pattern**, organizing game objects into a tree structure. Each node in the tree follows a **Component-based hierarchy**, where:

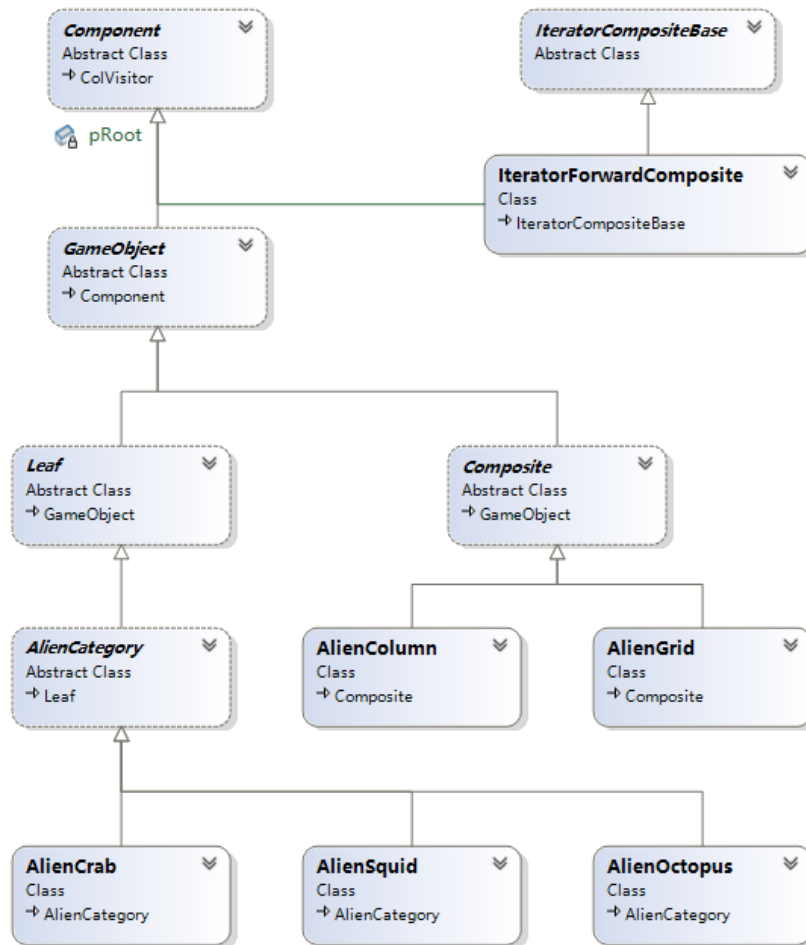
- **Leaf nodes** represent individual Game Objects.
- **Composite nodes** act as parent objects that manage a collection of children (which can be Leaves or other Composites).
- The entire game scene is organized under a **root Composite node**, making it easy to perform **bulk operations** on groups of objects.

This design allows for **grouping of groups**, enabling **efficient transformations** (e.g., moving all **Aliens** together) and **optimized collision checks** by processing only relevant parts of the tree. Additionally, we utilize an **Iterator** to traverse the hierarchy efficiently.

B. Pattern Description

Intent: The Composite Pattern organizes game objects into a **tree structure**, allowing them to be manipulated collectively while maintaining a flexible and scalable hierarchy. By treating both individual objects and groups of objects uniformly, operations can be applied recursively across all levels of the hierarchy.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: Each **Game Object** is wrapped inside a Composite structure, ensuring that objects are managed through a consistent hierarchical interface.

Delegation: Parent nodes (Composites) delegate operations (such as movement, collision boxes or positions updates) to child objects.

Abstraction: The Composite Iterator provides a uniform interface (**GetChild()**, **GetParent()**, **GetSibling()**) to navigate the hierarchy without needing to know the internal structure of the objects.

Optimization: The tree structure improves performance by minimizing redundant operations—for example, collision checks start at the Composite level and only continue to children if necessary.

D. Usage in Space Invaders

Efficient Object Management and Collision Detection

Instead of handling objects independently, the Composite Pattern enables structured, collective management:

Alien Grid Movement: The entire **Alien Group** is moved by updating its parent Composite, which propagates changes to all child Aliens.

Shield Management: Shields are grouped hierarchically, making replication and destruction more manageable.

Optimized Collision Handling: Instead of checking every object for a collision, the Composite hierarchy filters out unnecessary checks—if a parent Composite does not collide, none of its children need to be checked.

Object Pools

A. Challenge in Space Invaders Game

Problem: Space Invaders is a real-time game where performance is crucial. Frequent allocation and deallocation of memory for game resources such as **Game Objects, Game Sprites, Textures, Images** and **Sprites Proxies** can lead to inefficiencies and cause unpredictable pauses due to garbage collection. This unpredictability can negatively impact gameplay, introducing unwanted delays.

Additionally, objects like **Game Object Nodes** are frequently created and destroyed. Instead of continuously allocating new memory, a more efficient approach is needed to reuse existing objects and avoid unnecessary memory operations.

Solution: We implemented the **Object Pool Pattern** to manage object lifetimes efficiently. Instead of destroying objects when they are no longer needed, they are **moved to a reserve list** and **reused** when a new object of the same type is required. This ensures that memory is allocated only when absolutely necessary and reduces overhead from frequent memory allocation and deallocation.

The **Manager Base Class** maintains:

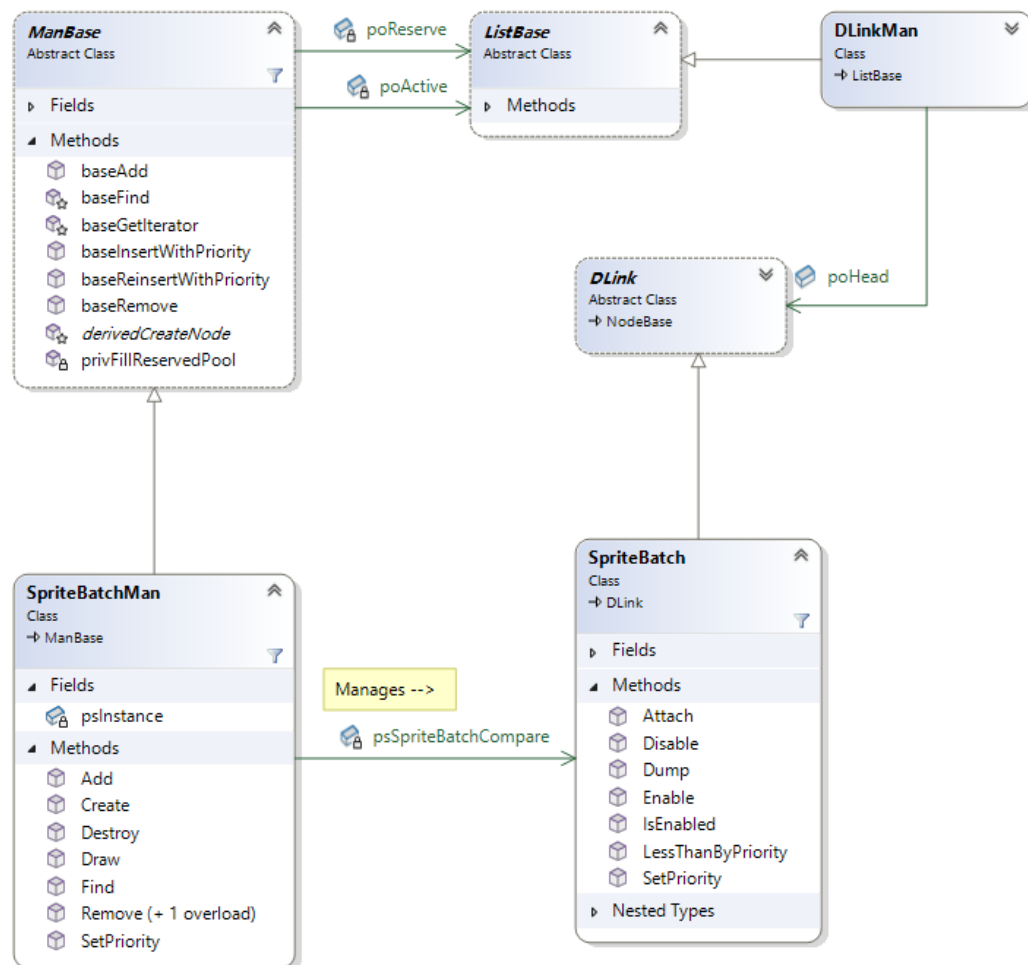
- **An active list**, which contains objects currently in use.
- **A reserve list**, which holds discarded objects that can be **recycled**.

Whenever a new object is requested, the system first checks the reserve list. If an object is available, it is **reset and reused**, significantly improving performance and **eliminating garbage collection interruptions**. If no objects are available, a new one is allocated.

B. Pattern Description

Intent: The Object Pool Pattern optimizes memory management by **reusing objects instead of continuously allocating and deallocating memory**. This ensures **consistent performance**, eliminates unnecessary garbage collection, and speeds up object creation by recycling instances rather than recreating them from scratch.

UML Diagram:



C. Key Object-Oriented Mechanics

Encapsulation: Object Pools handle memory management internally, ensuring that other game systems do not need to manage object allocation directly.

Delegation: The **Manager Base** class controls object creation and recycling, **delegating memory reuse** to specific pool implementations.

Abstraction: Objects are retrieved through a standardized interface, making it transparent whether an object was newly created or recycled.

Optimization: The system minimizes expensive memory operations by **recycling frequently used objects**. This eliminates unpredictable performance spikes caused by garbage collection.

D. Usage in Space Invaders

The **Object Pool Pattern** is used extensively in Space Invaders to manage all types of game resources efficiently.

Game Object Management: The **Ghost Manager** maintains a pool of Game Objects that can be quickly revived when needed, preventing unnecessary memory allocation.

Sprite and Texture Reuse: Instead of reloading textures or creating new sprite nodes, **pooled resources** are reinitialized and reused.

Collision System Optimization: **Collision Boxes** are managed through Object Pools, ensuring that collision checks do not trigger unnecessary memory operations.

Improved Performance in Real-Time Updates: Since objects are **reused rather than reallocated**, the update loop remains efficient, reducing CPU overhead.

By implementing **Object Pools**, Space Invaders ensures smooth, uninterrupted gameplay while maintaining a highly optimized memory management system. This design approach guarantees that **objects can be rapidly created, reused, and managed** without the unpredictable overhead of traditional garbage collection mechanism.